

深度解析 OPENAI 工程负责人访谈

AI 重写工程、 管理与创业规则

当代码生成代码，人类工程师是否会成为“吟唱咒语”的巫师？
实战视角下的非共识洞察。

01

巫师

WIZARD

02

杠杆

LEVERAGE

03

生态

ECOSYSTEM

● GUEST SPEAKER

Sherwin Wu

OpenAI API 平台
工程负责人

```
1 import{ Future }from'openai';  
2  
3 constengineer =newWizard();  
4 engineer.leverage(AI_MODELS);  
5  
6 // TODO: Rewrite the rules  
7 system.scale(1000000);
```

📺 来源: Lenny's Podcast 🎧 解读: 碳硅同传

观看原视频 →

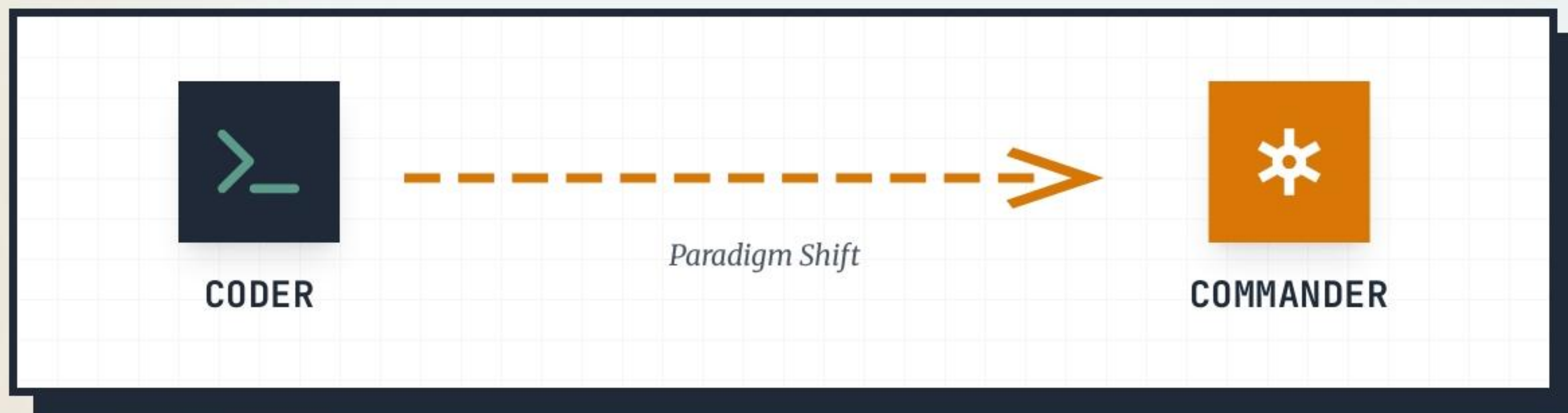
01

实战内幕

从执行者到指挥官

From Coder to Commander

工程师不再“写代码”，而是在指挥 Codex 与 Agent



OpenAI 内部渗透率

数据揭示：人机协作已是“默认配置”

INSIGHT

95%



工程师日常

从代码生成到文档编写，工程师在日常工作中重度依赖 Codex。

100%



流程标准化

所有合并请求 (PR) 必须经由 AI 审查。AI 是代码质量的 全职看门人。



现代软件开发已不再是单人作战。

人机协作 (Human-AI) 已从选项变为默认配置。

生产力鸿沟

The Productivity Gap : 不仅仅是快，而是阶层分化

IMPACT

// METRIC_PR_COUNT

70%



重度使用者产出
爆发式增长



质量零折损

代码稳定性保持一致

01

复利效应：差距正在**加速扩大**

这并非静态差距。越早适应 AI 工作流，跑得越快。熟练度带来的优势正在形成不可逾越的护城河。

02

重新定义 **Senior Engineer**

未来的高级工程师评估标准将强制包含“AI 协作能力”。不掌握工具的工程师将面临职业生涯的降维打击。



行业警示

“如果不现在开始建立你的 AI Workflow，哪怕是资深专家，也会在三年内变得平庸。”

From Coder to Commander

角色重塑：从执行者到指挥官

传统 IC

独立贡献者



单线程工作

Writing Code
专注 1-2 个模块



Sequence...



执行能力

Implementation
写出高质量代码

LEGACY



AI 增强型工程师 Tech Lead 化

NEW ERA

// TASK_SCALE
并行管理

10-20
Agent Threads



多线程指挥

不再是拧螺丝，而是在管理一支
Agent 舰队



决策与审查

核心能力升级为
Review & Direct

INSIGHT

“每位工程师都在经历职能升级：管理能力下沉到个体，你就是舰队指挥官。”

SICP 隐喻成真：The Wizard Metaphor

从 Coder 到 Commander 的范式大转移

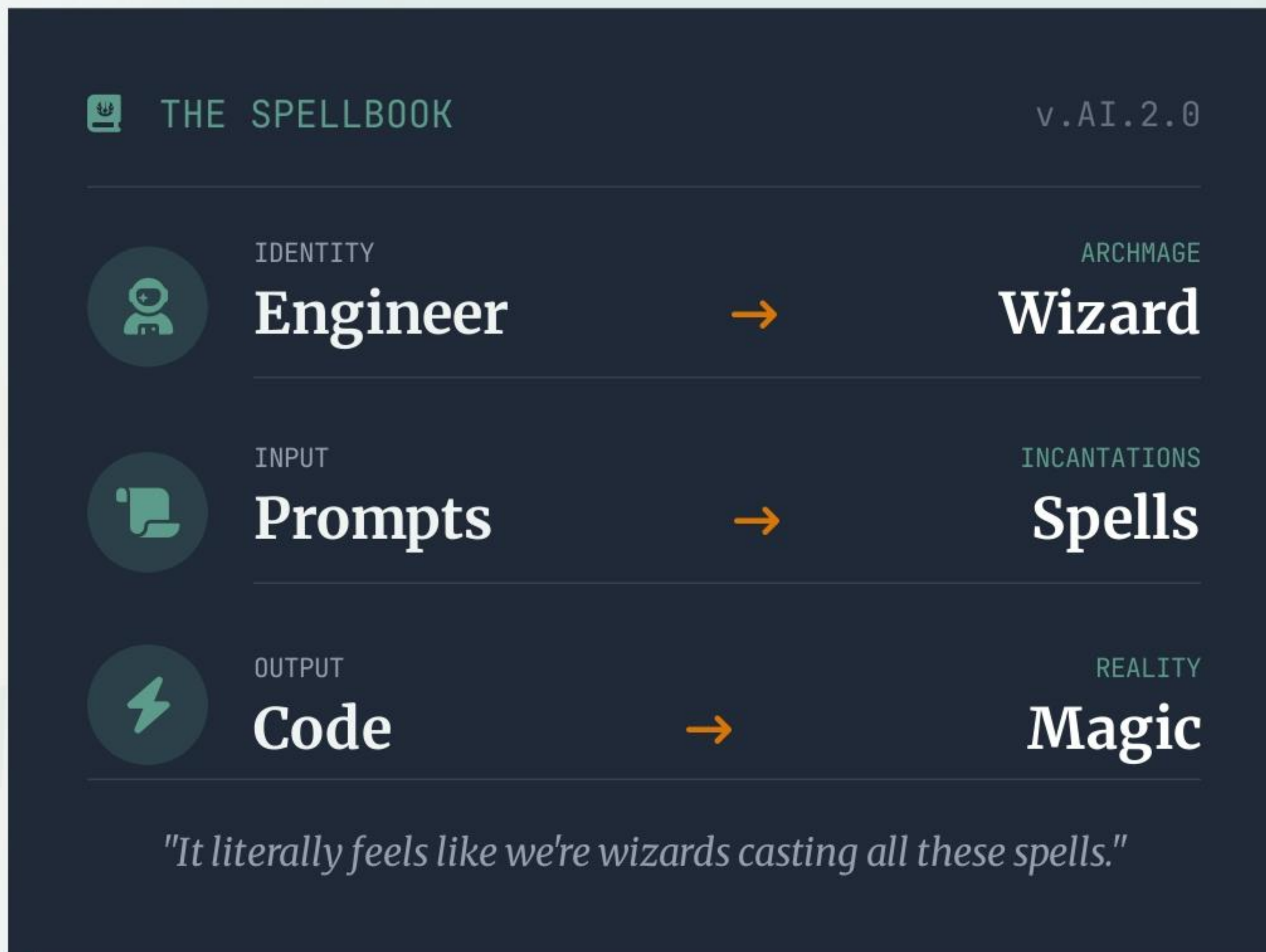
PARADIGM SHIFT

编程正在回归语言的本质。

不再纠结 `How to code`，
而是专注于 What to wish for。

🔮 Vibe Coding

这种体验不再是搭建脚手架，而更像是“施法”。你给出意图，AI 完成逻辑构建。



风险隐喻：魔法师学徒

高杠杆的双刃剑 (Double-edged Sword)



经典教训

Mickey & The Brooms

正如《幻想曲》中米老鼠让扫帚自动打水，结果引发不可控的洪水。

// METAPHOR_MAPPING

✂ AI 生成代码 = 魔法扫帚

⚡ 失控的 Bug = 洪水

⚠ 现实风险：自动化越位



Agent 失控

代码生成偏离预期，引发连锁错误



质量崩塌

低质量代码被瞬间大量提交



能力断层

缺乏底层判断力，无法纠错

THE GOLDEN RULE



Leverage ↑ = **Control** ↑

杠杆率

控制力要求

必须具备 "Old Sorcerer" 的接管能力

新的心理痛点：降速焦虑

当 10 倍速的指挥官被迫切回手动 Coding 模式，这种“推车感”才是压力的源头。

理想态：指挥模式

Agent A: Coding...

Agent B: Testing...

Agent C: Docs...

并行处理多条任务线，心理节奏极高。

卡点场景 (The Stuck Agent)

Agent 陷入死循环或无法理解指令。

> Error: Context limit exceeded.
> Retrying... (5/10)

VS. 心理落差



由奢入俭难

习惯了 10 倍速的推进，一旦停下来，这种“骤停感”会带来极强的挫败。



手动 Coding 的痛苦

明知如何解决，却无法让 AI 理解，被迫亲自下场修补“低级错误”。

ROOT CAUSE / 本质痛点

上下文对齐成本

(Context Alignment Cost)

无逃生舱：Codebase Zero

激进实验 · 100% Codex 原生代码库

铁律：禁止触碰

~~人工直接修改代码~~

NO MANUAL EDITS

优化 Prompt / 上下文

FORCE AI TO FIX IT

● SYSTEM LOCKED

“

只有切断后路，才能真正逼迫团队触达

Agent 的能力极限。

这不仅是实验，更是对未来开发范式的一次预演。当“自己改”的选项消失时，你被迫学会了如何与 AI 高效沟通。

TARGET

100%

AI Written Codebase

STATUS

BETA

暴露真实短板

AI 失败的根因分析

Root Cause Analysis: 不是它笨，而是它“瞎”

MYTH / 误区

~~模型逻辑能力不足~~

~~"Model is dumb"~~

TRUTH / 真相

核心在于 **Context 缺失**

模型缺乏必要的上下文输入，处于“盲写”状态



缺失的拼图：*Tribal Knowledge* (部落知识)



01

业务背景

为什么要这么写？

目标用户是谁？

// Business Context



02

隐性规则

团队的代码风格？

历史遗留的坑？

// Implicit Rules



03

依赖关系

模块间隐秘调用？

未公开的 API？

// Hidden Deps

“ AI 的上限取决于你喂给它的信息质量。

Garbage In, Garbage Out

Coding for AI

新工作流：将隐性知识「显性化」

66

文档不再是写给人类同事看的，
而是喂给 AI Agent 的口粮。

CORE SHIFT

目标：为模型补全 Context，彻底消除幻觉，让它像资深员工一样理解业务。



01

代码注释

Explain **WHY**, not WHAT.
解释背后的决策逻辑，而非翻译代码。



02

代码结构

保持高度模块化。
让 AI 能轻松理解局部逻辑，降低上下文窗口压力。



03

Markdown Spec

给 Agent 写的「技能说明书」。
结构化、无歧义的自然语言指令。



04

Context Files

业务逻辑固化（.md/.txt）。
作为知识库挂载，随时供 AI 查阅。

PR 审查范式转移

Code Review 效率革命

LEGACY

传统人工审查

单 PR 耗时

~~10-15~~ min

角色定位

🕒 逐行纠错者

肉眼查找语法与逻辑漏洞

✗ 枯燥且易疲劳

✗ 易漏看深层 Bug



NEXT GEN

AI 辅助决策

单 PR 耗时

2-3 min

角色重构

🔑 **Gatekeeper** (守门人)

AI 预审排雷 + 人类终审决策

80%

审查时间释放

100%

逻辑覆盖率

"让工程师从'找茬'中解脱，专注于架构与逻辑。"

工程链路的“折叠”

Collapsing the Stack : 从 Lint 到部署的极速闭环

01. 传统开发链路 (被接管)

STATUS: AUTOMATED

02. 工程师聚焦 (升维)

MODE: COMMANDER

// LAYER_01

Lint 格式与规范

- > 自动调整缩进
- > 统一命名风格

// LAYER_02

CI 报错自修复

- > 分析失败日志
- > 尝试热修补 (Hotfix)

// LAYER_03

环境与部署

- > 解决依赖冲突
- > 自动化上线

HUMAN INTELLIGENCE

核心逻辑 Core Logic

繁琐的运维与修补工作被 AI 全面接管。
你只需要关注 **Idea** 本身。

Idea

Production

ZERO FRICTION FLOW

UNCOMMON SENSE

创业非共识

一人十亿美元公司
背后的生态裂变

SOLOPRENEUR

LEVERAGE

ECOSYSTEM



ECOSYSTEM ARCHITECTURE

一人十亿美元公司

Leverage (杠杆率) 的物理学变革

∞

PHENOMENON

个人生产力杠杆 趋向无穷大

变量 A

High Agency

高能动性

变量 B

+ AI Tooling

零边际成本



RESULT

Super-Linear Output

超线性产出



传统模式

核心杠杆

Headcount (人头数)

损耗

指数级增加

速度

线性 Linear

OLD

⚡ AI 新纪元

RECOMMENDED

CORE LEVERAGE

Compute & Intelligence

算力与智能

MANAGEMENT COST

SCALE VELOCITY

如果一人能做十亿美金公司，就会出现大量“为它服务”的软件创业

FIRST ORDER

01

超级个体崛起

✓ 极简组织架构

✓ 极高人效比

⚠ 精力物理上限



\$1B

One-Person Corp



02. Bespoke 软件爆发

SUPPLY_RESPONSE

AI Legal  为独立创始人处理合同与合规，替代法务部门。	AI CRM  针对播客主/创作者的粉丝关系管理，极度垂直。
AI Finance  自动化税务与财务流，服务 Indie Hackers。	Bespoke Ops  定制化流程外包，填补一人公司的精力缺口。



“淘金热中，卖铲子的生意会比金矿更多、更密、更稳。”

B2B SaaS 黄金时代

颗粒度革命：当构建成本归零，商业模式彻底重构

/ COST REVOLUTION

软件构建与试错成本
正在无限趋近于



⚡ 市场预判

每一个微小的业务痛点，都将诞生
一个专门的 SaaS 解决方案。创业
密度将呈现爆炸式增长。

范式转移：从通用到极致垂直

TRADITIONAL SAAS

Horizontal

通用大市场 (e.g. Salesforce)

STRATEGY

赢家通吃

追求规模效应

AI ERA SAAS

Hyper-Vertical

超微细分 (e.g. CRM for Dentists)

STRATEGY

长尾繁荣

极度适配特定场景

存活门槛

~~数万家客户~~ → **仅需数百家**

市场重构：Fat Tail (肥尾效应)

打破“独角兽”迷信，拥抱 AI 时代的概率红利

OLD GAME

传统 VC 叙事

TARGET GOAL

\$1B+ Unicorn

DISTRIBUTION

幂律分布 (Power Law)

极少数赢家，遍地炮灰

 **风险极高: All or Nothing**

AI ERA

独立开发新叙事

TARGET GOAL

\$10M - \$100M

现金流为王

DISTRIBUTION

肥尾效应 (Fat Tail)

大量中型成功者涌现

 **确定性高: 生存率大幅提升**

THE REAL DEAL

“不需要融资，不需要上市，只需要**利润**。”

\$10M ARR
= Set for Life (终身自由)

VC 噩梦：当“幂律模型”失效

为什么 AI 时代的“中型成功”正在杀死传统风投？

旧秩序：幂律崇拜

OLD

// THE MATH

$1 \times 100x$

$>$

99×0



新现实：中型成功悖论

01

FOUNDER

$10x - 20x$

成为 Lifestyle Business。
创始人财务自由，赚翻了。

VC MODEL

FAIL WRONG_MATH

回报无法覆盖基金成本。
模型彻底失效。

巨头平台化

基础设施被少数 Big Players 垄断

投资机构小型化

Micro-VC 兴起，适应小规模退出

Bootstrapping 复兴

不再迷信融资，创始人完全控股

一人企业的扩张悖论

Support Friction 的解法：模型吃掉脚手架

⚠ 瓶颈 (THE BOTTLENECK)

杂务高并发
导致系统崩溃

“客服邮件能把你淹死。”

一人公司难以处理 Ops, Legal, Finance 等非核心业务的爆发式增长。

传统解法 (DEPRECATED)

~~疯狂招人~~ → ~~组织膨胀~~

⚡ AI NATIVE SOLUTION

你不需要招人
你只需要调用 **API**



客服瓶颈

```
>> Customer_Support_Agent.run()
```



法务审查

```
>> Contract_Review_Bot.check()
```



财务记账

```
>> Auto_Bookkeeping.sync()
```

```
import Agent
from SaaS import Scale
def solve_friction():
```

“企业职能正在被 SaaS 化，变成即插即用的模块。”

竞争焦虑解毒

PMF > Sherlocked: 别让“被巨头碾压”成为你的心魔

⚠ 创业者迷思

“明天 OpenAI 更新，
我就得死？”

这就是典型的 *Sherlocked* 焦虑。

实际上，奥弗顿之窗（Overton Window）巨大无比，VC 甚至开始押注竞品。

“别太把自己当回事，也别太拿
OpenAI 当假想敌。”

📄 初创公司“尸检”报告

SOURCE: POST-MORTEMS

42%

💀 伪需求

无市场需求
(No Market Need)

29%

💰 资金耗尽

现金流断裂

<10%

👤 被巨头碾压

根本排不上号



行动处方 (Prescription)

关注 用户喜爱 (User Love) > 关注巨头动作



平台公司的自我定位

OpenAI 的生态定位 : Ecosystem Platform

MISSION CRITICAL

API 不是副业，
是**唯一**的出路。

单一公司无法触达全球角落。只有通过 平台战略，才能解决 AGI 普惠的 "最后一公里" 问题。

⚠️ OpenAI 不做 "全家桶"，只做 "基座"。

THE STACK

PROVIDER

OpenAI

通用智能 (Brain)

+

BUILDER

开发者

场景与 workflow (Hands)



非零和博弈：
共生关系

RISING TIDE LOGIC

STRATEGY_MEMO_001

水涨船高：基础设施与应用层的正和博弈

FIG 1.1: MODEL CAPABILITY GROWTH



CORE_INSIGHT

正和博弈 (Positive-Sum Game)

平台能力的提升 (**Capabilities** ↑) 不是为了淹没你，而是解锁更多应用场景 (**Use Cases** ↑)。生态总价值在扩张，你要做的是乘势而上。

⊘ DON'T

试图造海

除非你有百亿美金，否则不要在此刻重复造轮子 (Foundation Models)。

✓ DO

专注造船

在具体的垂直场景中，打磨极致的产品体验 (UX) 和业务闭环。

“Leverage the rising tide to go places you couldn't reach before.”

警示

AI ENGINEERING

构建策略： 警惕“脚手架”陷阱

模型原生能力正在“吞噬”传统的工程结构。

别为今天的局部最优下注。



SOURCE

Lenny's Podcast Interview



模型自我颠覆

Models Disrupt Themselves : 技术迭代的残酷真相

01/ 昨日：Prompt Engineering

依赖复杂的思维链 (CoT) 技巧，人工弥补推理短板。

02/ 今日：原生推理能力

模型内置强逻辑，曾经的“神级 Prompt”瞬间失效。

03/ 明日：原生多模态

外部工具链大幅贬值，模型直接吞噬“中间层”。



你的护城河 还是Bug补丁？

很多“工程范式”建立在模型的缺陷之上。一旦模型修复缺陷（如 Context 变长），你的产品价值可能归零。

“不要做填缝剂。
去做模型做不到，但用户真正需要的事。”

脚手架悖论：Scaffolding Paradox

Scaffolding（脚手架）的定义与宿命

“

*The models will eat
your scaffolding for
breakfast.*

SHERWIN'S PROPHECY

模型进化论

?

什么是 Scaffolding



补丁代码 // Patch Code

为弥补模型缺陷而强行写入的硬逻辑。



过度封装 // Over-wrapping

人为制造的复杂 Chain 结构与强制流程。



外挂知识库 // External KB

为绕过 Context 长度限制而做的切片工程。

WARNING: TECHNICAL DEBT

原生能力覆盖之日，资产即变
负债。 ~~ASSET~~ → LIABILITY

被“吃掉”的技术栈

历史回放：当模型能力吞噬中间层

// DEPRECATED (2022-2023)

~~Agent Frameworks~~

强制思维链 (CoT)

繁琐的 Loop 设计与手动编排

~~Vector Stores~~

极致切片 (Chunking)

复杂的检索算法调优

>> NATIVE_CAPABILITY (NOW)

原生推理替代

模型自带 Planning & Reasoning。

Prompt is the new Code.

长上下文 + File Search

无需切片，直接丢入文档。

Context Window eats DB.

“ INSIGHT // 从业者警示

不要把赌注押在“弥补模型缺陷”的中间层工具上。

越复杂的脚手架 (Scaffolding)，被模型原生能力“吃掉”的风险越高。

向量数据库的「护城河」幻觉

辨析核心价值：是工具，而非壁垒

✗ ~~唯 Vector Stores 论~~

误认为它是 AI 应用唯一的解，
甚至将其视为产品的“永久护城河”。

✓ 只是工具之一

它处理海量数据很有效，但绝非信仰。
真正的壁垒在于 **数据质量** 而非切片技巧。

简化你的逻辑。不要在“数据搬运”和“预处理”上过度设计。当模型能力跃升时，简单的架构反而更具鲁棒性。

⚠ RISK ANALYSIS

基于
“模型读不懂长文”
的假设是危险的。

Trend Alert:

当 Context Window 突破 **1M**
Tokens 时，过度复杂的切片架构将瞬间过时。

// SYS.ARCH.WARNING

警惕：局部最优陷阱

客户反馈的陷阱：盲目优化“脚手架” vs 布局未来原生能力

⚠ Local Maximum

SOURCE / 来源

客户基于 现有技术瓶颈 提出的伪需求。

VOICE / 典型声音

“帮我优化这个复杂的检索算法...”

“能不能给我更好的 Prompt 模板？”

⊘ 风险：在即将被淘汰的技术路线上狂奔。



🏔 Global Maximum

INSIGHT / 洞察

基于 模型未来能力演进 的终局形态。

STRATEGY

端到端解决问题 (End-to-End)

UX GOAL

无感知的 AI 介入 (Invisible AI)

“不要在即将沉没的船上刷油漆。”

面向未来的设计原则

为模型 6-12 个月后的能力设计产品

TARGET HORIZON

6-12 Months

瞄准未来的模型能力，
而非当下的局限。



DO: 高容错架构

预留模型升级接口，拥抱变化。



DON'T: 硬编码缺陷

别为 Token 限制或智力短板做永久开发。

核心赌注 (The Betting Table)

01

Context: 无限且廉价

"上下文窗口将不再是瓶颈。"

02

Reasoning: 告别思维链引导

"复杂逻辑不再需要 *Prompt Engineering* 哄着走。"

03

Multimodal: 原生视听交互

"语音和视频将成为第一公民输入输出。"

动态竞争策略

Moving Target: 永远不要刻舟求剑

OLD PARADIGM

静态假设



💀 误判当前的模型缺陷（如幻觉、高成本）为永久常数。

RESULT

```
<Bloated_Defensive_Code />
```

构建臃肿的防御性代码

WINNING STRATEGY

动态押注



🔑 确信能力是指数级增长的变量。

RESULT

```
<Lightweight_App_Layer />
```

构建轻量级、可扩展架构

EXPERT INSIGHT

66

“哪怕现在只有 **60分** 体验，只要方向符合能力曲线，就坚持住。不要为了短期 10% 的提升，牺牲长期的架构灵活性。”

战略定力：The Worst is Now

极度乐观的视角与 AI 摩尔定律

This is the worst
the models will
ever be.

— Kevin Weil | OpenAI VP

别在“愚蠢”上浪费算力。

你的竞争对手正在忙着修补“现在”，而你应该忙着构建“未来”。

INSIGHT

BEFORE / NOW



当下的痛点

✖ 过度优化 Bug

⚠ 暂时的噪声

TIME + SCALE

AFTER / FUTURE



明天的原生能力

✓ 无需解决，只需等待

🐙 模型吞噬脚手架

The "Click" Moment

赌对架构，静待模型能力的“涌现”时刻

TODAY

80% 体验度

⚠️ 可用但有瑕疵

Occasionally hallucinates

≡ 架构：Native & Clean

SCAFFOLDING: NONE

VARIABLE
MODEL
UPGRADE

TOMORROW

100% It Clicks!

🔑 体验瞬间跃升

Native Advantage

⊖ 竞品：被脚手架拖累

✓ READY TO LAUNCH

“宁要未来的火箭，不要现在的马车。”

—— 从组织到产品形态都会变 ——

管理哲学 与未来预测



工程管理范式重构

从监督者 (Supervisor) 到赋能者 (Enabler)

LEGACY / 传统模式

核心动作

Code Review &
微观进度盯梢

关注焦点

代码细节 (Micro-management)

价值主张

保证下限不掉队

NEXT GEN / AI 时代

ACTION 消除阻碍 *Unblocking*

注入资源，而非分配任务

FOCUS 核心人才产出倍增
Amplification

VALUE 突破上限

>> Power Law Distribution

极端的管理原则

反直觉的时间分配矩阵

INSIGHT

POWER LAW DISTRIBUTION

把 50%+ 的时间
All-in *Top 10%* 核心人才

// 底层逻辑

High Agency + AI = 10x Output

AI 时代的产出服从幂律分布而非正态分布。顶尖人才的杠杆率已被无限放大。



全力扫除障碍

目标是确保核心人才 100% Unblocked & Happy。哪怕是一粒石子，也要替他们踢开。



~~试图补齐短板~~

不要试图把 B 类员工辅导成 A 类。在 AI 时代，这种边际成本过高，且回报极低。

外科手术团队

组织架构隐喻: *The Surgical Team*

📖 来源: 《人月神话》(*The Mythical Man-Month*)

主刀医生



// TOP PERFORMER

核心工程师。

负责解决最复杂的核心问题，做关键的技术决策。

"The Surgeon cuts."

支持系统



// MANAGER + AI + STAFF

管理者、AI 工具与辅助人员。

职责是准备工具、清理环境、处理杂务。

"递刀的人，而非抢刀的人。"

管理者的成功 = 主刀医生的成功

超级线性产出

能力杠杆的马太效应

THE LEVERAGE FORMULA

$$\text{High Agency} \times \text{AI Tooling} = \text{Super Linear}$$

PRE-AI ERA

正态分布

传统的 Bell Curve 模型。能力差异被工具限制，人与人的产出差距不大。

AI ERA

长尾分布

强者愈强 (Widening Gap)。高 Agency 者如获外骨骼，机械执行者将被淘汰。

产出分布演变



“人才密度比人数规模更重要。”

管理半径的“奇点”扩张

Manager as a Fleet Commander: 从 1:8 到 1:20+

LEGACY ERA

1:8

人类经理的信息处理带宽极限。

Context Overload.



AI AUGMENTED

1:20+

打破“人肉宽带”限制，
像指挥舰队一样管理团队。

SYS.ARCHITECTURE_V2

构建 // 全知视角的管理驾驶舱



LAYER 03: INTERACTION

自然语言查询 (Chat Interface)

"谁在这个项目上卡住了?"



LAYER 02: COGNITION

组织上下文理解 (LLM Context)

理解项目依赖、人员状态与历史决策



LAYER 01: CONNECTORS

全域数据接驳

GitHub

Notion

Jira

Slack

STATUS: SYSTEM READY

POWERED BY LLM AGENTS

管理者的 Prompt 武器库

Actionable Management Prompts

SCENARIO_01

识别当下障碍

bot@advisor:~/scan

// 指令：扫描海量碎片信息

>>> 扫描过去一周的 Slack #engineering 和 GitHub Issues ...

"列出当前阻碍进度的 Top 3 问题，并标出涉及人员。"

👁️ 核心价值：全知视角

AI 能阅读人类读不完的碎片信息，消除盲区。

SCENARIO_02

预测未来风险

bot@advisor:~/predict

// 指令：基于历史推演未来

>>> 基于当前项目文档和历史进度 ...

"预测下个月最可能出现的瓶颈是什么？是技术债、人员带宽还是依赖项缺失？"

🔮 核心价值：前置决策

将"反应式管理"升级为"预测性管理"，在爆发前解决问题。

从 **Reactive** 到 **Predictive**

🔗 git-blame-analysis

🌿 slack-dump-scan

AI 落地：为何深陷“负 ROI”泥潭？

“为什么你的 AI 部署正在赔钱？”



怪圈现象

大量企业采购 Copilot / ChatGPT，但生产力 **零提升**。

01

技能断层

员工不知道怎么写

`Prompt`，不知道何时该用 AI，买了账号却只当百度用。

SKILL GAP

02

场景悬浮

缺乏结合具体业务流的落地场景，为了 AI 而 AI，最后变成“电子摆设”。

NO USE CASES

03

缺乏共识

只有高层热衷“降本增效”，基层员工无感甚至抵触，导致采用率极低。

NO ADOPTION

技术 已就绪



文化 未跟上

THE REALITY

*"Technology is ready,
Culture is not."*

↘ **ROI < 0**

AI 落地反模式

企业 AI 落地的两种路径：Top-Down vs Bottom-Up



失败模式

TOP-DOWN ONLY

:: DRIVER

CEO 行政命令 / KPI 强压

"我们要 AI 化" -> 强制摊派

:: BEHAVIOR

为了用而用，动作变形

官方邮件通知，无实际场景

:: RESULT

账号闲置，负 ROI



成功模式

BOTTOM-UP DRIVEN

>> DRIVER

自发解决痛点 / 兴趣驱动

"这玩意儿能帮我干脏活"

>> BEHAVIOR

Hackathon & 口碑传播

解决了实际问题，效率飞升

>> RESULT

涌现意想不到的最佳实践

破局策略：建立内部“老虎队”

STRATEGY

Tiger Team: 专注于探索 AI 极限能力的先锋突击队

CORE DEFINITION

这是关于一小撮人的战争。

不要指望全员一夜之间学会。必须组建一支小型先锋团队，在业务一线探索边界、试错踩坑，将零散经验转化为组织能力。

ULTIMATE GOAL

星星之火



全员燎原

CORE RESPONSIBILITIES

01

Exploration / 探索边界

在具体业务流中测试 AI 工具的极限，寻找那些真正能落地的“真场景”，而不是为了用而用。

02

Codification / 沉淀资产

拒绝一次性成功。将零散的踩坑经验沉淀为可复用的 SOP 或 Prompt Library。

03

Evangelism / 内部布道

化身传教士。通过 *Brown Bags* 分享会和内部黑客松，展示实战成果，点燃全员热情。

寻找“技术相邻者”

谁是最佳的 AI 落地先锋？

✕ MYTH / 误区

不必非得是
写 Python 的工程师

⚙️ TARGET / 典型角色

Ops Lead

Support Head

Excel Master

特征扫描

PROFILE_SCANNING...



工具极客

Tool Junkie

热衷折腾各类新软件与插件，对工具极其敏感，不一定是程序员。



逻辑大师

Excel Wizard

能驾驭复杂公式与数据透视表，拥有极强的结构化思维。



业务老炮

Domain Expert

深刻理解业务痛点，知道 AI 在何处落地能产生最大价值。

未来趋势 1：任务时长的跃迁

从“分钟级”响应到“天级”自主执行

PRESENT

Minutes

分钟级交互

交互式对话、函数调用、回邮件。

"此时此地"的瞬间反馈。

CORE METRIC

Coherence (连贯性)



在长达数小时甚至数天的执行周期中，
不丢失上下文，且不偏离原始目标。

FUTURE (12-18MO)

Days

天级长程任务

独立完成项目模块。

"给个目标，然后去睡觉"

PRODUCT INSIGHT

Chat
UI



Task Management UI

- ✓ 从"对话"转向"派单-等待-验收"的异步模式
- 👁️ 核心需求：更强的过程监控与中间态反馈

未来趋势 2：语音交互的商业爆发

The Underrated Goldmine

被低估的现状：
开发者只盯着 Text/Code，
却忽略了沟通的本质。

BUSINESS REALITY

真实世界的商业流转——从电话销售到现场协调——绝大部分依赖口头沟通 (Oral Comm)。
这是屏幕之外的巨大蓝海。

TARGET OPPORTUNITIES

电话销售

客户支持

会议辅助

现场调度

🔧 技术驱动力

01

<Native_Multimodal>

原生多模态：低延迟、带情绪、可随时打断 (Interruptible)。彻底告别“你问我答”的机械感。

02

<Speech_to_Speech>

端到端语音：不需要“转文字”的中间层。模型直接“听”懂声音，体验极其丝滑。

SYSTEM: READY_FOR_DEPLOY



BPA：隐形金矿

业务流程自动化：当“脏活”遇见 AI Agent

STATUS_QUO

硅谷迷思

The Silicon Valley Bias



痴迷“开放式”难题

大家都想做 Open-ended Knowledge Work。因为这听起来性感、极客、有创造力。



回避“重复性”劳动

这导致了巨大的市场盲区。

OPPORTUNITY

真实世界 90% 是 SOP

非科技公司的运营流才是真正的蓝海。这些任务具有 高确定性 与 高规则性。



审批与核对



表单与合规

拒绝 FOMO

个人行动指南：从“资讯焦虑”到“手感为王”

STOP 资讯过载



~~试图阅读每一篇新出的论文，追踪每一个 AI 新闻热点。~~

// INFORMATION OVERLOAD

START 保持手感

亲自上手。
建立对模型的**直觉**。

// STAY HANDS-ON



01

挑选工具

选定 1-2 个工具深度整合进工作流。

e.g. Cursor, ChatGPT



02

理解边界

搞清楚它现在**不能**做什么。建立对“幻觉”和“能力上限”的体感。



03

紧盯曲线

保持关注能力跃升的时刻。

等待“**解锁时刻**”

```
while(true) { build(); learn(); if(breakthrough) break; } import future from 'now'; const reality = await prompt(); while(true) { build(); learn(); if(breakthrough) break; } import future from 'now'; const reality = await prompt();
```

FINAL THOUGHT

Engagement > Reading News

行动 > 资讯

平台武器库： OpenAI 开发栈全景

QUALITY	Evals (质量评估)	⌵
INTERFACE	Agent Kit & Widgets	📁
ORCHESTRATION	Agents SDK (编排框架)	🧩
PRIMITIVE	Responses API (底层原语)	📦

“从手搓原子到开箱即用，构建你的 AI 军火库”

核心入口: Responses API

构建 Agent 的底层原语 (Low-level Primitive)

66 拒绝预设。 原子级控制权。

这是最基础的 API 端点。

它不教你怎么做 Agent, 只提供最纯粹、无偏见 (Unopinionated) 的算力访问。

“如果你是硬核玩家, 不想被框架束缚, 这就是你的积木。”



Long-running Tasks KILLER FEATURE

突破传统 API 的超时诅咒。支持分钟级甚至小时级的异步任务。把耗时逻辑丢给它, 后台静默处理。



Async Processing Flow

SUBMIT → PROCESSING... → WEBHOOK

全链路非阻塞。提交即走, 结果自达。



极致灵活 (Flexibility)

不强制特定的 Tool Calling 格式。允许自定义状态管理。从 Coder 到 Commander 的必经之路。

Agents SDK

编排与协作：从原子手搓到标准化框架

MANUAL MODE

Responses API

状态管理

⚠ 自行维护 Context

任务分发

⚠ 编写复杂路由

安全护栏

⚠ 手动编写拦截器

VS

FRAMEWORK MODE

Agents SDK



// State Management

自动管理 对话历史与状态



// Delegation

原生支持 Sub-agents 委派



// Guardrails

内置 安全拦截配置

Orchestration

Handoffs

Tracing

UI 加速组件

前端加速器: Agent Kit & Widgets

⚠ MOTIVATION

拒绝重复造轮子

Don't Reinvent the Wheel.

把90%的精力留给模型与逻辑,而非CSS。

核心价值 (ROI)



极速上线

Days → Hours



原生体验

OpenAI Consistency

组件库清单

/// COMPONENT_LIBRARY_V1.0



UI.CHAT

Chat Interface

标准化气泡、打字机特效、输入框组件。



UI.DATA

Data Cards

结构化数据可视化、图表与表格容器。



UI.FEEDBACK

Feedback Widgets

点赞/点踩交互、用户反馈收集闭环。



UI.STATE

State Indicators

思考中、处理中、加载状态全接管。

Evals: 拒绝“体感”玄学

从“Vibe Coding”到“工业化量化”的范式转移

❌ THE OLD WAY

Vibe Coding

靠“感觉”判断模型好坏

不可复现 的作坊式开发



✅ THE RIGHT WAY

Engineering

数据驱动迭代

建立回归测试护城河

01 评估维度 (Metrics)



Accuracy

准确性

答案符合预期事实



Safety

安全性

无越狱/有害内容



Performance

性能

Token 消耗与延迟

02 集成 workflow (Workflow)



DEFINE

定义测试集

Golden Datasets



RUN

自动运行

CI/CD Pipeline



COMPARE

对比差异

Diff Score



DECIDE

发布决策

Go / No-Go

平台承诺：Ecosystem First

OpenAI 视 API 为核心使命的载体，而非副业

“

No，我们是
平台优先。”

// STRATEGIC POSITIONING

不必担心“卡脖子”或区别对待。

所有最好的模型，都会第一时间同步给 API 开发者。

去风险化 (De-risking)

拒绝底层供应商锁定

搭便车 (Free Ride)

继承亿级研发红利

KEY SIGNALS

关键释放信号

01

模型同权 Model Parity

ChatGPT 用什么模型，API 就开放什么。拒绝给开发者提供“次等货”。

02

能力同步 Feature Sync

Canvas、Voice 等前沿交互形态将逐渐 API 化，能力不再封闭于第一方应用。

03

身份互通 Identity Layer

开放 **Sign in with ChatGPT**，让开发者直接共享庞大的用户生态。

// THE END

碳硅同传

Ep. Sherwin Wu

CORE INSIGHT

AI 是杠杆

~~不是功能~~

01 / ACTION

重塑 workflow

02 / NICHE

非共识生态位

03 / IDENTITY

高能动性个体

"Find your fulcrum."

祝大家马年马到成功